Visual intuition (for media / AI)

For media team (also can be used as AI prompt): need to insert image here, as if `bob.eth` (Bob) is associated with stealth address named `0xStealth` in some registry. From this stealth address, other addresses branch off in different directions: `0xabc...`, `0xdef...`, `0xfe3...` and so on, say, 8-10 of them (draw an anonymity mask next to each address near its `0x...`). It's as if Bob can secretly control them using private key for `0xStealth`. It's as if every time he gets paid he creates new address `0xfe1` (random every time) under mask that derives from `0xStealth`. You could draw Alice (alice.eth) next to Bob and Alice paying on 0xfe1... instead of just "payment".

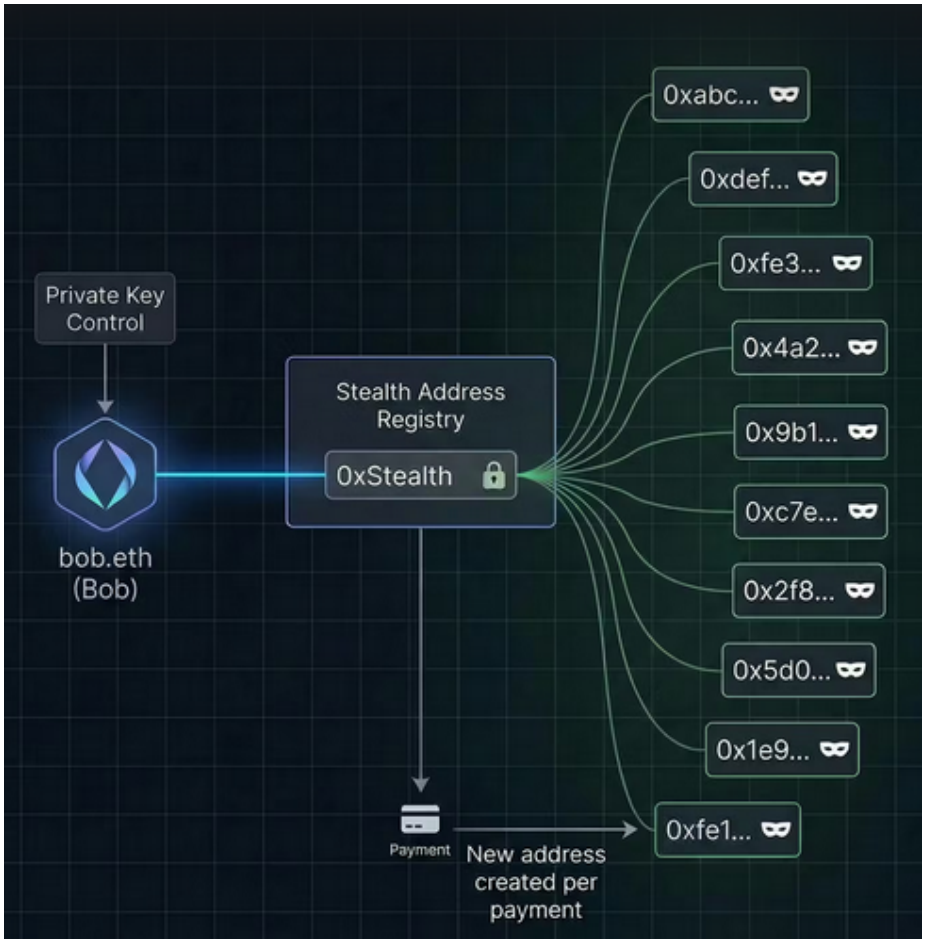

Figure 1: image0

Short article introduction / Summary

This article discusses how to implement private transactions based on ERC-5564: Stealth Addresses on any blockchain that supports secp256k1 cryptography (VARA, BTC, ETH), and how we could use Vara Network as storage for dApp that implements private transactions. Unlike mixers like Tornado Cash, ERC-5564 was designed to hide on-chain behavior, not for laundering.

Implementing private transactions on Vara Network & Ethereum, how our chain could be used as layer for storage

Recently, privacy coins reached another ATH. We were curious about what we could offer regarding private transactions and whether we could help other blockchains use Vara Network as blockchain for low-cost data storage. We have something to offer our users!

Introduction to Elliptic-curve Cryptography (ECC) without understanding all math We have 3 elements: - Private keys or scalars

They are usually indicated in lowercase letters:

$k = 123$ (private key)

$p_{smth} = 456$ (private key)

In our example we say that let's say $k = 123$, but in reality private keys are random number from `1` to `1157920892373161954235709850086879078528375642790749043826051631415181614943337` (number with 78 digits).

- Public keys or points on elliptic curve with coordinates (x, y):

  There is well-known point $G(x, y)$ or generator point G and its coordinates are known to everyone.

  There is multiplication operation $P(x, y) = k \cdot G$ as result of which you get point $P(x, y)$ on curve by multiplying private key k by generator point G.

  Usually, private keys are denoted by lowercase letters, and public keys derived from private keys are denoted by uppercase letters:

  $k = 123$ (private key)

  $K = k \cdot G$ (derive public key)

  $p_{smth} = 456$ (private key)

  $P_{smth} = p_{smth} \cdot G$ (derive public key)

  Also coordinates (x, y) are usually omitted since it is usually clear that this is point on curve by first uppercase letter.

  You can also add public keys or points on an elliptic curve:

  $R(x, y) = P(x, y) + Q(x, y)$ (addition of points)

or just

$R = P + Q$ (addition of points)

- Blockchain addresses (`0x123...`) derived from public keys:

  Let's say you have public key P_{smth} derived from private key p_{smth}, you can get address on blockchain usually via $H(P_{smth})$, where $H(P_{smth})$ is hash function such as `SHA256` or `KECCAK256`.

  For example, Ethereum simply writes point (x, y) in bit representation and then uses `KECCAK256(x || y)` and then throws away first 12 bytes of hash function.

  $p_{smth} = 456$ (private key)

  $P_{smth} = p_{smth} \cdot G$ (derive public key)

  $a_{smth} = H(P_{smth}) = KECCAK256(P_{smth})$ (derive address)

Some properties of ECC: - ECDH protocol:

Two parties Alice and Bob can calculate some secret point $S(x, y)$:

Alice:

$a = 123$ (private key)

$A = a \cdot G$ (derive public key)

Bob:

$b = 456$ (private key)

$B = b \cdot G$ (derive public key)

Alice sent Bob public key A (e.g. on public forum website)

Bob sent Alice public key B (e.g. on public forum website)

ECDH formula:

$S(x, y) = a \cdot B = b \cdot A = a \cdot b \cdot G$

Now look at $a \cdot B$. It's same as $a \cdot b \cdot G$ since $B = b \cdot G$. Same thing works in reverse.

This means that Alice and Bob, by exchanging public keys on public forum, can calculate secret point known only to them. They could then take x coordinate of this point and use it as shift for cipher (for example, Caesar cipher) or use any other more modern one with password x. They could then communicate privately with each other on forum.

- Linearity in Elliptic Curve (EC) scalar multiplication

  We won't prove anything and will write this as property of ECC:

  $(k + m) \cdot P = k \cdot P + m \cdot P$

Introduction to ERC-5564: Stealth Addresses (Private, non-interactive transactions) Let's say Alice wants to send Bob 1 ETH. They may also have domain names on blockchain.

Alice	—[1 ETH]–>	Bob
`0xaaa`		`0xbbb`
`alice.eth`		`bob.eth`

If they do this directly, everyone will see on blockchain "Alice (`0xaaa`) pays Bob (`0xbbb`) 1 ETH once a month". This is bad, because as users migrate from Web2 to Web3, it will be possible to track on-chain user behavior and habits.

Many mixers try to hide sender's address (Alice) and break transaction graph, but what if we tried to hide recipient's address (Bob)? This is also good because source of funds is visible and you can always prove to exchange that it is legitimate. Even though source of funds will be visible, this does not violate privacy of users, and we will soon see why.

Stealth Addresses work like this:

1. Bob generates 2 private keys (spend and view):

   p_{spend}

   p_{view}

   Bob derives 2 public keys from private keys:

   $P_{spend} = p_{spend} \cdot G$

   $P_{view} = p_{view} \cdot G$

   Using 2 public keys, Bob calculates his stealth address (66 bytes):

   `st:eth:0x034f6340cfdd930a6f54e730188e3071d150877fa664945fb6f120c18b56ce1c090201b0906..`

   As you can see, address has special prefix `"st:<chain>:"`, which indicates that this is stealth address

   Actual format is `st:<chain>:0x<compressed spendPublicKey><compressed viewPublicKey>`

   Bob goes to special smart-contract called "registry of stealth addresses" (ERC-6538: Stealth Meta-Address Registry) and says:

   - bind my address (`0xbbb`, `bob.eth`) to `"st:<chain>:0x034f6340..."`

2. Alice opens dApp and clicks "send 1 ETH to `bob.eth`"

   Domain system tries to find `bob.eth` and resolves it as `0xbbb` address

   dApp goes to "registry of stealth addresses" again and tries to find stealth address for `0xbbb` address and finds `"st:<chain>:0x034f6340..."`

Part with domains and registering in "registry of stealth addresses" is optional, and in fact Bob can just send his `"st:<chain>:0x034f6340..."` directly to Alice's DM. However, it is convenient way for Bob to say on-chain "anyone can send me money privately".

Alice generates an ephemeral private key:

$p_{ephemeral}$

Alice derives public key from private key:

$P_{ephemeral} = p_{ephemeral} \cdot G$

Alice parses format `st:<chain>:0x<compressed spendPublicKey><compressed viewPublicKey>` and extracts Bob's public keys:

P_{spend}

P_{view}

Now Alice (and Bob too) can calculate shared secret using ECDH protocol:

$s = p_{ephemeral} \cdot P_{view}$

Usually, in ECDH protocol, y coordinate is discarded and only x coordinate is used, so we will do same ($s = x$ is shared secret known only to Alice and Bob).

Since s is a well-known x coordinate on curve and can be brute-forced (in theory), hash of s is usually taken (known as HKDF):

$s_h = \mathrm{h}(s)$

Now Alice needs to get v (view tag). This is simply first byte of s_h and is used by Bob to optimize blockchain scanning (described later):

$v = s_h[0]$

Why do we take first byte, and how secure is this (especially considering that s_h will be used as part of Bob's private key)? Mainly because taking last byte would reveal whether number is even/odd.

Alice derives public key from s_h, which will be used on next step:

$S_h = s_h \cdot G$

Now Alice calculates $P_{stealth}$ and it will be new empty address (in more correct terminology, public key) with no history each time, and Bob will be able to get $p_{stealth}$ (private key of $P_{stealth}$):

$P_{stealth} = P_{spend} + S_h$

How exactly does Bob get $p_{stealth}$? This works because p_{spend} (private spend key) is only known to Bob, but not to Alice.

$p_{stealth} = p_{spend} + s_h$

Do you still have Linearity in Elliptic Curve (EC) scalar multiplication in mind? We need it now:

$p_{stealth} \cdot G$

or

$(p_{spend} + s_h) \cdot G = (p_{spend} \cdot G) + (s_h \cdot G) = P_{spend} + S_h$

In Vitalik Buterin's article this is called "key blinding mechanism". If $p_{stealth} = s_h$, then both Alice and Bob could compute private key. This means that Alice could make deposit to Bob's new empty address and immediately return it back. Therefore, we add p_{spend} (which only Bob has) to s_h and use Linearity property of Elliptic Curve (EC) scalar multiplication.

Now Alice needs to convert Bob's public key into an address:

$a_{stealth} = \text{pubkeyToAddress}(P_{stealth})$

When this algorithm completes, Alice will calculate following data:

- $a_{stealth}$ (new empty address Bob, each time different)
- $P_{ephemeral}$ (Alice's ephemeral public key)
- v (view tag, special byte for optimization on Bob's side)

Now Alice needs to publish this data in blockchain and also specify $schemeId = 1$, it is also worth noting that $metadata = v$. Alice calls `announce` method on `ERC5564Announcer` contract:

```
contract ERC5564Announcer {
    event Announcement(
        uint256 indexed schemeId,
        address indexed stealthAddress,
        address indexed caller,
        bytes ephemeralPubKey,
        bytes metadata
    );

    function announce(
        uint256 schemeId,
        address stealthAddress,
        bytes memory ephemeralPubKey,
        bytes memory metadata
    ) external {
        emit Announcement(
         schemeId,
         stealthAddress,
         msg.sender,
         ephemeralPubKey,
         metadata
       );
```

```
    }
}
```

As you can see, `ERC5564Announcer` contract doesn't do anything. It simply accepts data from anyone and emits event (saving this data on blockchain). In our implementation, we wrote our own version, `ERC5564AnnouncerWithHooks`, which also checks validity of `ephemeralPubKey` and that `metadata` is at least one byte long.

After calling `announce` method, Alice can make deposit of 1 ETH to $a_{stealth}$ address. If Alice is transferring ERC-20/ERC-721 tokens, Alice should also deposit small amount of native currency so that Bob has funds for withdrawal.

3. Bob doesn't initially know any $a_{stealth}$ address and needs to scan blockchain and find every `Announcement` there, and then apply some cryptography to it.

   Bob parses `Announcement` and gets following data (this data is stored in separate variables associated with `Announcement`):

   - $a_{stealth}$ (someone's stealth address, not necessarily Bob's)
   - $P_{ephemeral}$ (someone's ephemeral key, not necessarily Alice's)
   - v (view tag, special byte for optimization on recipient's side)

   Bob multiplies his p_{view} (private view key) by $P_{ephemeral}$ (Alice's ephemeral public key from `Announcement`) to calculate shared secret using ECDH protocol:

   $s = p_{view} \cdot P_{ephemeral}$

   Since s is a well-known x coordinate on curve and can be brute-forced (in theory), hash of s is usually taken (known as HKDF):

   $s_h = \mathrm{h}(s)$

   Now we will finally return to v (view tag) and consider why it is needed. This is simply first byte of s_h:

   $v = s_h[0]$

   Once Bob has calculated v (view tag) he compares calculated v with one provided in `Announcement` and if they don't match then this `Announcement` was not created for Bob and should be skipped.

   This way, Bob only needs to do single elliptic curve multiplication for each ephemeral public key to compute shared secret, and only 1/256 of time would Bob need to do more complex calculation to generate and check full address.

   Bob derives public key from s_h, which will be used on next step:

   $S_h = s_h \cdot G$

Bob computes stealth public key:

$P_{stealth} = P_{spend} + S_h$

Now Bob needs to convert public key into an address:

$a_{stealth} = \text{pubkeyToAddress}(P_{stealth})$

If Bob's calculated $a_{stealth}$ matches one specified in `Announcement`, then Bob can calculate private value of $a_{stealth}$:

$p_{stealth} = p_{spend} + s_h$

Once Bob has calculated private key, he can own ETH or ERC-20/ERC-721 tokens.

Finally, to make all formulas more visual, we've created table that fully explains math. You can refer to it to better understand process:

Alice	Bob
	p_{spend}
	$P_{spend} = p_{spend} \cdot G$
	p_{view}
	$P_{view} = p_{view} \cdot G$
$p_{ephemeral}$	
$P_{ephemeral} = p_{ephemeral} \cdot G$	
$s = p_{ephemeral} \cdot P_{view}$	$s = p_{view} \cdot P_{ephemeral}$
$s_h = \text{h}(s)$	$s_h = \text{h}(s)$
$v = s_h[0]$	$v = s_h[0]$
$S_h = s_h \cdot G$	$S_h = s_h \cdot G$
	$p_{stealth} = p_{spend} + s_h$
$P_{stealth} = P_{spend} + S_h$	$P_{stealth} = P_{spend} + S_h$
$a_{stealth} = \text{pubkeyToAddress}(P_{stealth})$	$a_{stealth} = \text{pubkeyToAddress}(P_{stealth})$

So why are these called "private transactions"? After reviewing technical details, you might be thinking, "but you're not hiding Alice's address". However, if Alice previously paid 1 ETH per month to Bob, everyone could see it and track Bob's behavior (for example, that he spends 0.1 ETH per month on trading on DEX). Now, everyone sees that Alice pays 1 ETH once a month to a new, empty address not directly associated with Bob. Alice could also split amounts, for example, by 2x 0.5 ETH and pay someone else once a month, not just Bob, in which case it would be difficult to tell any economic relationship between Alice and Bob. In turn, Bob, as soon as he receives 1 ETH, could create a new deposit address (so as to have no connection with his previous deposit addresses) on CEX and deposit it, then sell ETH for fiat currency. If exchange asks him about origin of funds, everything is visible on-chain. Everyone knows that Alice paid someone 1 ETH, but they don't know who exactly. This is an ideal solution for private donations or private receiving salaries.

Project status, does Gear Technologies have demo/dApp? Good news is that we've already written our own implementation of ERC-5564: Stealth Addresses in about ~2 days and put it in gear-foundation/stealth-addresses repository. Since Vara Network uses Substrate Framework, we also support secp256k1 cryptography and all this code can be ported without any changes to mathematics of working with elliptic curves.

We don't currently have dApp, but we have research and cryptography for it. Idea is that we could host `ERC5564Announcer` and `ERC6538Registry` contracts on Vara Networkand save users money on fees, since publishing data on our blockchain is very cheap. In turn, users can use any network (VARA, BTC, ETH, etc.) for private transactions. All they need to do is buy our token on one of CEXs for $5, and that will be enough for hundreds (maybe even thousands) of private transactions. You connect two wallets (Vara Network and Ethereum) to dApp and have choice: pay `ERC5564Announcer` fee on our chain or on Ethereum. But private transaction occurs, for example, in Ethereum network or any other EVM network, and our chain is used to store `Announcement` and index it in some wallet that supports ERC-5564. This is also cool because dApps don't need to have any centralized backend and can only use connections to blockchain nodes as static HTML page on IPFS.